

Decision-Making for Autonomous Driving with Multiple Experts via Skill Discovery and Transformer

Jing Meng

School of Mechanical Engineering
Beijing Institute of Technology
Beijing, China
3120220376@bit.edu.cn

Haoyang Wang

School of Mechanical Engineering
Beijing Institute of Technology
Beijing, China
15101179309@163.com

Yunlong Lin

School of Mechanical Engineering
Beijing Institute of Technology
Beijing, China
yunlonglin@bit.edu.cn

Chao Lu*

School of Mechanical Engineering
Beijing Institute of Technology
Beijing, China
Zhengzhou Academy Of Intelligent
Technology
Beijing Institute of Technology
Zhengzhou, China
chaolu@bit.edu.cn

Jianwei Gong

School of Mechanical Engineering
Beijing Institute of Technology
Beijing, China
Zhengzhou Academy Of Intelligent
Technology
Beijing Institute of Technology
Zhengzhou, China
gongjianwei@bit.edu.cn

Abstract—In the urban environment, intelligent vehicles need to operate multiple driving tasks to handle complex and uncertain scenarios. Currently, Rule-based methods struggle with complex scenarios, only handling specific or simple cases. Data-driven methods, while more flexible, are reliant on extensive data, which has poor generalization to unfamiliar scenarios. Although reinforcement learning (RL) has contributed to pioneering new ideas in autonomous driving, it struggles to converge and fails to develop effective strategies as the complexity of tasks escalates. To solve these problems, we propose a hierarchical decision-making framework, named Hierarchical Multi-expert Decision Transformer (HMDT), which combines skill discovery and Decision Transformer. As a classic hierarchical reinforcement learning method, skill discovery decomposes a complex task into multiple basic sub-tasks to reduce the complexity. Decision Transformer models the traditional reinforcement learning as a sequence modeling problem, which learns the driving strategies for the sub-tasks. The proposed framework is evaluated in a complex urban environment built in the CARLA simulator. The experimental results demonstrate that HMDT achieves higher completion rates in single tasks and performs better in complex traffic tasks compared to baseline methods.

Keywords—Intelligent vehicle, Mixture of Experts, decision-making in multiple tasks, skill discovery, Decision Transformer

I. INTRODUCTION

In recent years, with the development of artificial intelligence in autonomous driving, research on autonomous driving systems (ADS) has beyond simple scenarios. Because of the exponential increase in complexity with scenarios, it is difficult for the intelligent vehicle to reach its destination within the scenario composed of multiple simple tasks, such as going straight, changing lanes, etc. Therefore, it is crucial to develop an adaptive multi-task decision-making system.

Over the past two decades, rule-based methods have been applied to driving tasks in urban scenarios. Among the rule-

based methods, the hierarchical state machine decision system manually designed in the DARPA Urban Challenge [1] improves navigation by structuring processes into states and transitions. Generally, rule-based methods are reliable and easy to design. However, as the complexity of the scenario increases, it becomes impractical to handle all the situations, especially for corner cases.

Therefore, learning-based methods are proposed to solve these problems. As a data-driven learning-based method, Imitation learning (IL) [2-4] trains a deep network to learn driving policies directly from expert data, thereby eliminating the need for manually designed strategies. However, due to data limitations, IL has poor generalization to unfamiliar tasks. On the basis of Markov decision process, reinforcement learning (RL) [5-7] is proposed. Unlike IL, RL learns policies through the interaction with the environment. However, in the complex tasks, it struggles to converge and fails to develop effective policies. Furthermore, designing reward functions has been tricky for RL, when the question is complicated. To avoid these problems, hierarchical reinforcement learning (HRL) [8-10] decomposes problems into sub-problems to tackle the high complexity challenges. Duan et al. propose a framework [11] that decomposes driving strategies both vertically and horizontally, thereby significantly reducing the burden of designing the reward function. Although the framework effectively simplifies the decision-making problem, it still increases the instability of vehicle control due to the decoupling of lateral and longitudinal dynamics.

With the development of world models and large language models such as GPT, researchers try to apply Transformer [12] to ADS. To enhance the performance of RL, Xiao et al. incorporates the attention mechanism into the policy network and makes improvements to the experience pool [13]. Despite improving performances, they reduce the learning efficiency of networks. Generative world models learns spatiotemporal representations of the agent and environment by unsupervised methods, and generates policies to solve specific driving tasks [14-16]. However, they demand high hardware resources and training durations. Besides, the Transformer has been applied in RL. Chen et al. treat RL as a sequence modeling task and

*Corresponding Author

This work was supported by the National Science and Technology Major Project (No. 2022ZD0115503) and National Natural Science Foundation of China (No. 52372405).

developed the Decision Transformer (DT) [17]. By utilizing the causal masking mechanism, this system trains optimal policies and mitigate the issues of noise and overestimation that typically arise during the update process of conventional RL algorithms [18]. While it performs superior to traditional RL methods in specific or single tasks, its effectiveness may diminish in more complex environments.

To improve the performance of the driving policies and reduce the computational overhead, this paper proposes a hierarchical decision-making framework, the Hierarchical Multi-expert Decision Transformer (HMDT), which decomposes complex traffic tasks into multiple simple tasks using the skill discovery, then tackles each sub-task through learning driving strategies from expert demonstration using DT. Therefore, adaptive decision-making and control across multiple tasks are accomplished.

The remainder of this paper is organized as follows. Section II describes the multi-scenario adaptive decision-making system based on skill discovery and DT, including its algorithmic details. In Section III, an urban environment with multiple driving tasks is designed in an autonomous driving simulator, and the specific experimental protocols are detailed. In Section IV, this paper presents a comparative analysis of the experimental results, comparing the proposed method with other baseline methods. Finally, Section V concludes the work and outlines directions for future research.

II. DECISION-MAKING FRAMEWORK WITH SKILL DISCOVERY AND DECISION TRANSFORMER

A. Problem Formulation

The decision-making in autonomous driving is modeled as Markov decision process (MDP) in this work. MDP is described by the tuple (S, A, P, R) , representing the state space, action space, state transition probability, and reward function, respectively. The trajectory is represented as:

$$trajectory = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T) \quad (1)$$

The model generates actions based on the expected future returns. Therefore, the policy return at timestep t is defined as

$U_t = \sum_{t'=t}^T r_{t'}$. The training data for the autoregressive model is modeled as:

$$\tau_T = (U_1, s_1, a_1, U_2, s_2, a_2, \dots, U_T, s_T, a_T) \quad (2)$$

DT trains an model $\pi(\theta)$ using the Transformer network, and is updated using Stochastic Gradient Descent (SGD). The loss function is used as:

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_i (a_i - \pi(\tau_{i-1} | \theta))^2 \quad (3)$$

where N represents the sample size of mini batch. a_i represents the target action.

B. Hierarchical Decision-making Framework

As shown in Fig. 1, this paper proposes a hierarchical architecture based on skill discovery and DT to achieve decision-making for intelligent vehicles in multiple tasks. The system is a hierarchical framework, with the lower layer for policy execution and the upper layer for expert network switching.

1) *Policy execution layer*: The driving policy for the intelligent vehicle is obtained through offline training using the DT architecture, which is able to abstract the traditional reinforcement learning problem as a sequence modeling problem, leveraging the sequence modeling objective to collect scenario data experiences and train a Transformer model.

2) *Expert network switching layer*: By recognizing environmental information for task decision-making, The agent constructs a series of skill options in complex tasks, with the implementation of skills carried out by the policy execution layer. Upon completing the execution of the previous option, the agent proceeds to execute the next option, continuing until the target destination is reached.

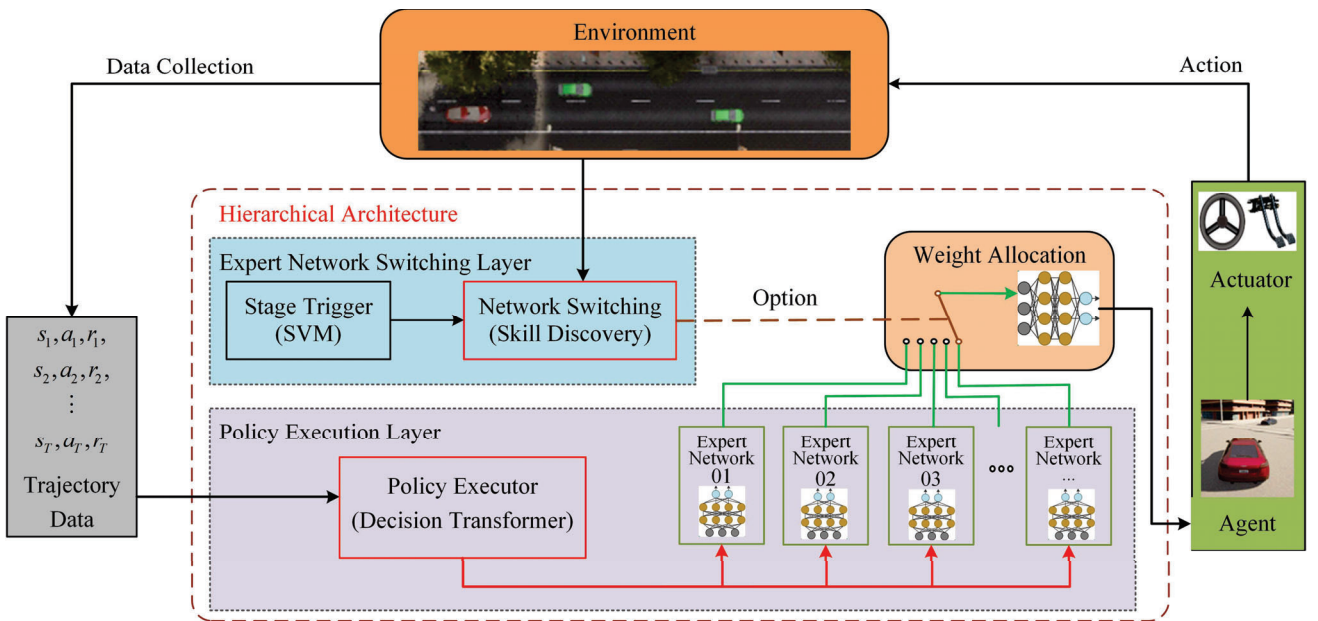


Fig. 1. The combined architecture based on skill discovery and Decision Transformer.

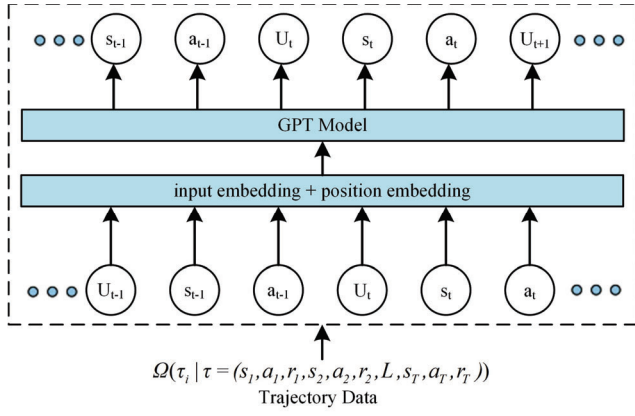


Fig. 3. The Decision Transformer architecture.

C. Policy Execution Layer

DT directly fits the optimal actions using a causal masking transformer. It is trained using randomly sampled training data by DT, including learning from a diverse set of experiences. During the testing phase, the model outputs the optimal policy by fitting the actions to achieve the maximum likely return. The GPT architecture [19] is used to train the policy of intelligent vehicles, as shown in Fig. 2.

D. Expert Network Switching Layer

Skill discovery introduces the concept of options, which distinguishes it from MDP and SMDP [20]. The option models with MDP at the lower level to train DT, and at the higher level, it abstracts and decomposes complex tasks into a chain of individual tasks.

We define an option using a triplet $\langle \mathcal{I}, \pi, \beta \rangle$, including an initial set $\mathcal{I} \subseteq \mathcal{S}$, an internal policy $\mu: \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, and a termination set $\beta: \mathcal{S}^+ \rightarrow [0, 1]$. The option is triggered only when $s_t \in \mathcal{I}$. If the option is triggered, it will take actions by policy π until it terminates on the termination set β . Fig. 3 shows the process of the skill discovery, which enables intelligent vehicles to adjust driving strategies in the complex tasks to reach the target destination.

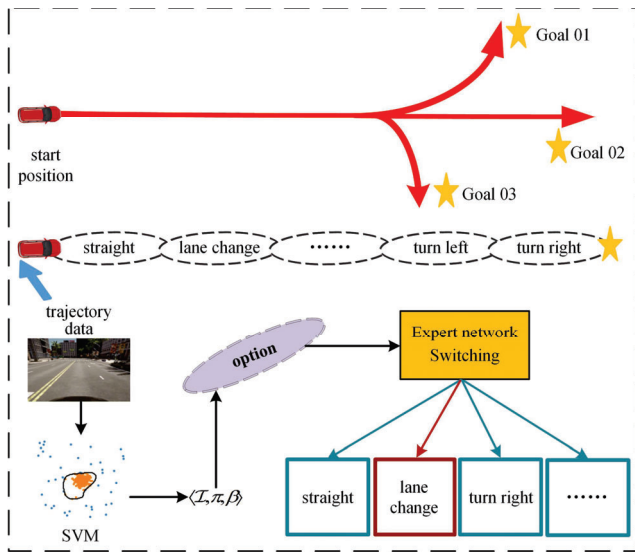


Fig. 2. The process of skill discovery.

E. HMDT Algorithm

HMDT algorithm is proposed to solve the global problem. by using the data from the K timesteps before t as input: $(U_{t-K}, s_{t-K}, a_{t-K}, \dots, U_{t-2}, s_{t-2}, a_{t-2}, U_{t-1}, s_{t-1}, a_{t-1})$, the model outputs the action at t . **Algorithm 1** describes the offline training process of the model.

Algorithm 1: HMDT Algorithm

Input: Prediction horizon K , the training datasets of all the target return U_θ .

Output: The action token at timestep t a_t .

Model Training Phase

- 1 choose the dataset of the subtask D_i ;
- 2 Initialize policy model $\pi_i(\theta)$;
- 3 **for** the trajectory data in D_i **do**
- 4 Compute the action token a using $\pi_i(\theta)$;
- 5 Compute the loss L and parameter θ updates;
- 6 **end for**

Model Evaluation Phase

- 7 Observe the initial state s_0
- 8 Initialize environment with (U_0, s_0)
- 9 **while** True **do**
- 10 Choose the expert network $\pi_i(\theta)$ by $s_t \in \mathcal{I}$
- 11 Compute the action token a_t by $\pi_i(\theta)$;
- 12 Take action token a_t , observe s_{t+1}, r_{t+1} ;
- 13 Update the input tokens $(U_{t-K}, s_{t-K}, \dots, U_t, s_t)$;
- 14 **if** done **do**
- 15 break;
- 16 **end if**
- 17 **end while**

III. EXPERIMENTS

A. Simulation Environment and Tasks

This paper conducts experiments and validations on the proposed algorithm framework in the CARLA simulator, which is an open-source autonomous driving simulation platform that provides a realistic and reliable simulation environment [21].

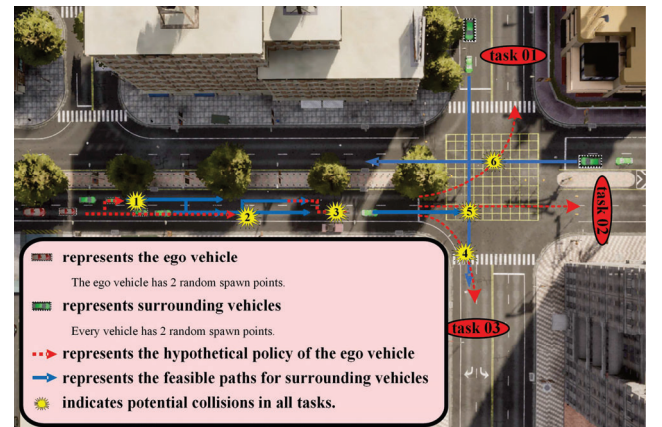


Fig. 4. The experimental environment built in CARLA.

Fig. 4 shows three complex tasks including going straight, turning left and right, which are composed of multiple simple tasks. The ego vehicle is represented in red, and surrounding vehicles are denoted in green. In these tasks, the ego vehicle is required to interact with the surrounding environment in real-time to ensure safety and adherence to traffic regulations, while navigating from the initial position to the target area as quickly as possible. The driving strategies for the ego vehicle may encompass a range of possibilities, such as driving straight, changing lanes, and making left or right turns. Due to the stochastic nature of the starting positions and real-time speeds of other vehicles in the environment, the strategies adopted by the ego vehicle may vary.

The ego vehicle is controlled by the proposed algorithm in the simulation environment. In contrast, the surrounding vehicles interact with the environment and generate control strategies using the Traffic Manager (TM) model integrated within the CARLA simulator.

B. Data Collection

The dataset comprises sample data generated by the random policy and expert data generated by the Logitech G29 driving simulator [22] operated by human drivers.

The data collected consists of a trajectory sequence τ for each episode. The modeling of the state space, action space, and reward function is as follows:

1) *State space*: We simplify the complex urban traffic scenario to the interaction process between the intelligent vehicle and other vehicles in the environment, ignoring road structures and other detailed information. The state space is defined as:

$$\begin{aligned} S &= [s_{host} \parallel s_{other} \parallel s_{waypoint}] \\ s_{host} &= [x_{host}, y_{host}, yaw_{host}, speed_{host}] \\ s_{other} &= [x_{other}, y_{other}, yaw_{other}, speed_{other}] \end{aligned} \quad (4)$$

where s_{host} denotes the state of the ego vehicle. The operator $\parallel$ denotes concatenation operation. s_{other} denotes the state of the other vehicle interacting with the ego vehicle. $s_{waypoint}$ denotes the state of waypoints on the lane. x, y, yaw are the pose of the vehicle and $speed$ denotes the speed of the vehicle.

2) *Action space*: DT directly learns basic actions from driving data, so the action space is represented as:

$$\begin{aligned} A &= [a_{throttle}, a_{brake}, a_{steer}] \\ a_{throttle}, a_{brake} &= [0.0, 1.0] \\ a_{steer} &= [-1.0, 1.0] \end{aligned} \quad (5)$$

where $a_{throttle}, a_{brake}, a_{steer}$ respectively represent the throttle and brake position, as well as steering angle.

3) *Reward function*: The reward function fully considers comfort, safety, and traffic efficiency during the driving process. It is specifically divided into speed reward, survival reward, safety reward, and destination reward. After extensive testing, the following reward functions have shown better performance:

$$\begin{aligned} R &= R_{speed} + R_{living} + R_{safty} + R_{goal} \\ R_{safty} &= -500 \text{ if collision} \\ R_{speed} &= \begin{cases} -1, & \text{if speed} < 2 \\ -2, & \text{if speed} > 8 \\ 0, & \text{else} \end{cases} \\ R_{living} &= -0.5 \\ R_{goal} &= 1000 \end{aligned} \quad (6)$$

where R_{speed} encourages the driving policy at speeds allowed by urban traffic. R_{living} is good at finding optimal strategies. R_{safty}, R_{goal} helps the agent avoid dangerous actions and reach the destination.

C. Offline Expert Policy Training for Sub-tasks

We select mini batches of sequences from the dataset with a length of K . The model is trained to predict actions using mean-squared error as the loss function.

In this phase, we train expert policies for four simple sub-tasks: going straight, changing lanes, and making left and right turns. The models are tested online in respective tasks each time the data increases by 10k samples. To validate the model's performance, the agent will iterate through 100 epochs in a randomly generated environment where the starting points of vehicles are randomly generated, and the TM model controls their driving strategies.

The hyperparameters for the model in the experiment are shown in the following table:

TABLE I. HYPERPARAMETERS OF DECISION TRANSFORMER

Hyperparameters	Value
Number of layers	3
Embedding dimension	256
Batch size	128
Dropout	0.1
Activation function	ReLU
Learning rate	5×10^{-5}
Weight decay	10^{-4}

D. Expert Network Switching for All Tasks

In complex urban traffic tasks, The skill discovery is employed to make decisions on expert driving strategies for multiple sub-tasks within each task.

The option space O is determined by the number of offline trained expert networks, which in this experiment can be represented as:

$$O = [o_{straight}, o_{lane_change}, o_{turn_left}, o_{turn_right}] \quad (7)$$

where $o_{straight}, o_{lane_change}, o_{turn_left}, o_{turn_right}$ represent four expert driving networks learned during the offline training phase: driving straight, changing lanes, turning left, and turning right, respectively.

The initial and termination sets are learned using a Support Vector Machine (SVM) classifier designed to solve potential non-stationary classification problems in continuous spaces. The labels are manually annotated.

E. Baseline Methods

In this work, The Decision Transformer algorithm without hierarchical structure is selected as the flat policy method. This method shares the same structure and hyperparameters with the policy execution layer in the hierarchical framework proposed in this paper.

IV. EXPERIMENTAL RESULT AND ANALYSIS

The experiment is divided into offline expert networks training for sub-tasks and testing with all tasks.

A. Offline Expert Policies Training for Sub-tasks

Fig. 5 illustrates the average return and scenario success rate across four simple tasks during offline training. Fig. 5(a) shows that, with the increase of the trajectory data, DT is able to learn the driving strategies for these tasks effectively. In the simplest straight-driving scenario, the success rate during the testing phase approaches 100%. The success rate of the lane-changing scenario consistently remains above 90%. As for the left-turn and right-turn tasks, the success rate generally reaches over 80%.

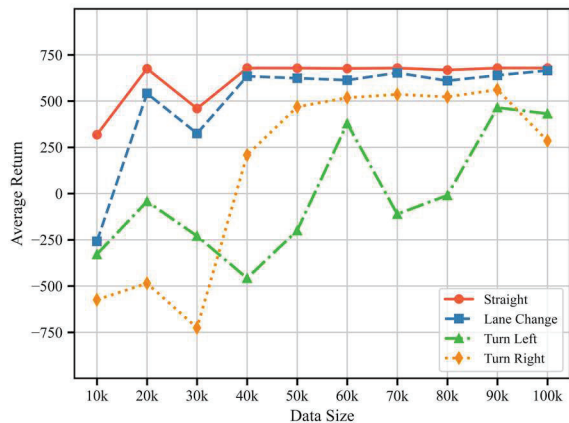
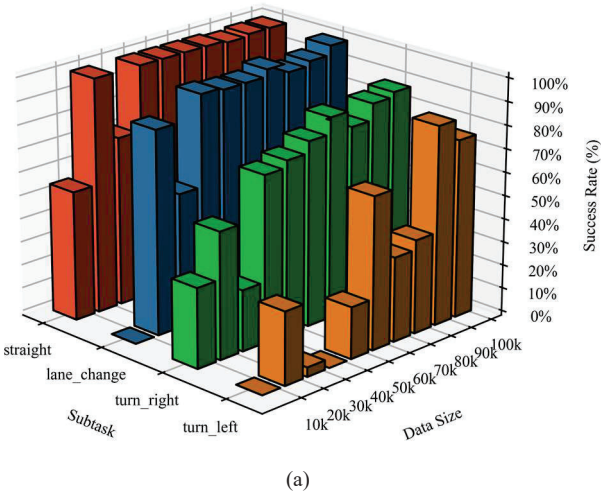


Fig. 5. The success rate and average return of subtasks. (a) illustrates the success rate of 100 epochs of testing in random environment. (b) illustrates the average return for the agent in testing phase.

The trend of the average return, as shown in Fig. 5(b), is similar to the success rates. It is important to note that this enables DT to acquire learning capabilities.

B. Testing with All Tasks

Table 2 shows the success rates of HMDT and the baseline method across all task tasks.

Method	HMDT	DT
Task 01	85%	35%
Task 02	92%	62%
Task 03	86%	44%

Compared to the baseline method, the proposed method exhibits high task completion rates across all tasks. Specifically, the success rate of Task 02 approaches 100% due to its relatively simple nature, which only involves straight driving and lane-changing strategies. Although Task 01 and Task 03 are more complex than Task 02, their success rate remains high. In contrast, the flat policy experiences a significant drop in success rates under complex scenario tasks. To visually represent the situation of the intelligent vehicle in the environment, Fig. 6 shows the snapshots of Task 03.

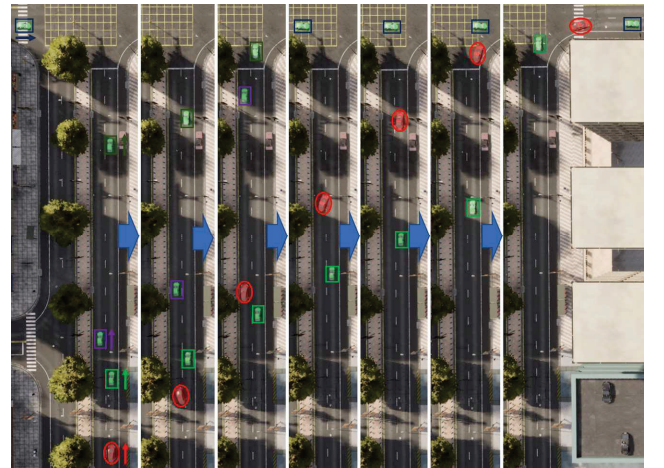


Fig. 6. Screenshots of Task 03 in the CARLA simulation.

The vehicle kinematic parameters are shown in Fig. 7. As is shown in Fig. 7(b), in Task 01, the vehicle ahead of the ego vehicle goes straight at a much higher speed than the vehicle in the left. The ego vehicle decides to go straight, rather than immediately changing lanes to the left. Only after overtaking the other vehicle and approaching the intersection makes the ego vehicle decision to change lanes to the left, and then turn left. Turning left, the ego vehicle makes a yielding decision to oncoming traffic. In Task 02, when the ego vehicle is at the intersection, it also makes a yielding decision to cross the vehicle. In Task 03, due to the slow-moving vehicle ahead of the ego vehicle, the ego vehicle decides overtake to maximize the potential reward. Meanwhile, as shown in the Task 03 curve of Fig. 7(a), after turning right, the ego vehicle made the decision to follow the preceding vehicle to avoid a collision.

V. CONCLUSION

This work proposes a hierarchical decision-making framework with good performance through complex driving tasks. The proposed framework integrates DT for training the offline expert driving networks in sub-tasks and the skill

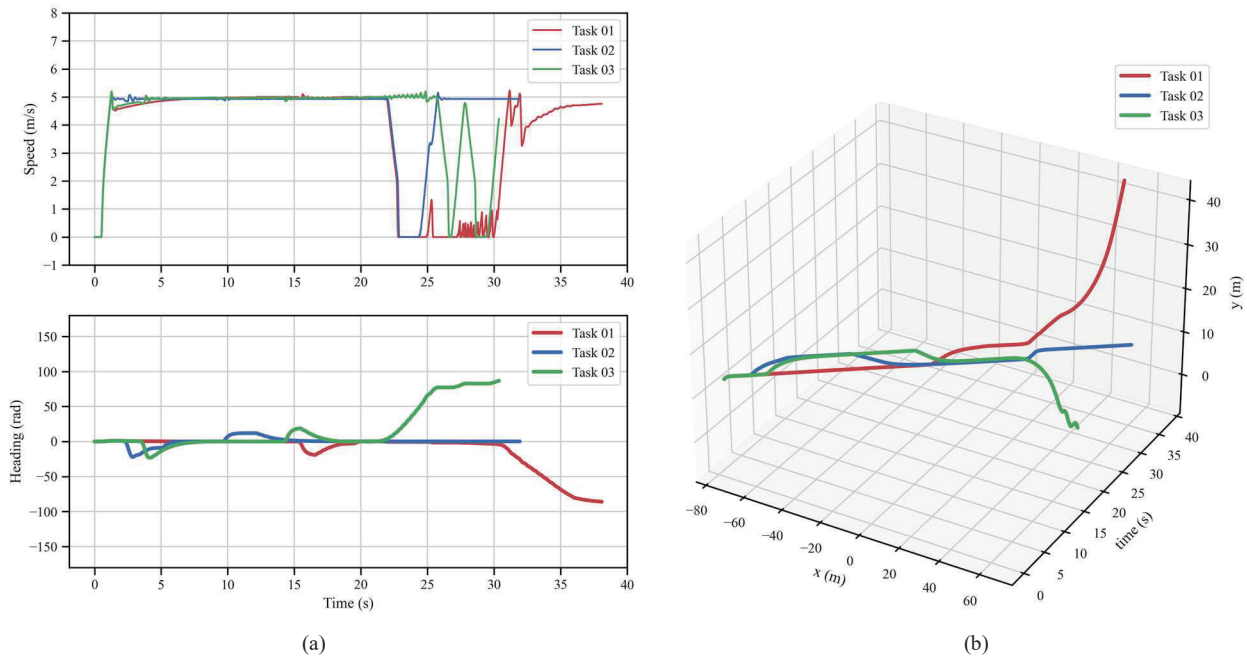


Fig. 7. The ego vehicle kinematic parameter of all tasks. (a) illustrates the velocity profile and the variation in heading angle of the ego vehicle, as depicted within the world coordinate system. (b) illustrates the spatiotemporal trajectory of the ego vehicle in the world coordinate system.

discovery method for switching the expert driving networks, enabling rapid adaptation for intelligent vehicles across complex tasks. The proposed method is validated in the CARLA simulator and compared with the baseline methods. Experimental results demonstrate that the proposed method performs better. However, there are also areas for improvement in this work. The reward function for RL and the policies of skill discovery are empirically designed, which makes it difficult for the model to achieve globally optimal. In the future, we will refine the proposed framework to achieve better performances in complex driving scenarios.

REFERENCES

- [1] M. Buehler, K. Iagnemma, and S. Singh, "The DARPA urban challenge: autonomous vehicles in city traffic," vol. 56. *Springer Science & Business Media*, 2009.
- [2] M. Bojarski *et al.*, "End to End Learning for Self-Driving Cars." *arXiv:1604.07316*, 2016.
- [3] F. Codevilla, M. Müller, A. López, V. Koltun and A. Dosovitskiy, "End-to-End Driving Via Conditional Imitation Learning," *2018 IEEE International Conference on Robotics and Automation (ICRA)*, Brisbane, QLD, Australia, 2018, pp. 4693-4700.
- [4] A. Hu *et al.*, "Model-Based Imitation Learning for Urban Driving," *Advances in Neural Information Processing Systems*, 2022, 35: 20703-20716.
- [5] P. Wang, C.-Y. Chan, and H. Li, "Automated Driving Maneuvers under Interactive Environment based on Deep Reinforcement Learning." *arXiv:1803.09200*, 2018.
- [6] J. Chen, B. Yuan, and M. Tomizuka, "Model-free deep reinforcement learning for urban autonomous driving," in *2019 IEEE intelligent transportation systems conference (ITSC)*, IEEE, 2019, pp. 2765-2771.
- [7] C. S. Arvind and J. Senthilnath, "Autonomous RL: Autonomous Vehicle Obstacle Avoidance in a Dynamic Environment using MLP-SARSA Reinforcement Learning," in *2019 IEEE 5th International Conference on Mechatronics System and Robots (ICMSR)*, Singapore: IEEE, May 2019, pp. 120-124.
- [8] C. Lu, H. Wang, C. Lv, J. Gong, J. Xi, and D. Cao, "Learning driver-specific behavior for overtaking: A combined learning framework," *IEEE Transactions on Vehicular Technology*, 2018, 67(8): 6788-6802.
- [9] Y. Yu, C. Lu, L. Yang, Z. Li, F. Hu, and J. Gong, "Hierarchical reinforcement learning combined with motion primitives for automated overtaking," in *2020 IEEE Intelligent Vehicles Symposium (IV)*, IEEE, 2020, pp. 1-6.
- [10] X. He, L. Yang, C. Lu, Z. Li, and J. Gong, "Adaptive Decision Making at the Intersection for Autonomous Vehicles Based on Skill Discovery," in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, IEEE, 2022, pp. 2842-2847.
- [11] J. Duan, S. Eben Li, Y. Guan, Q. Sun, and B. Cheng, "Hierarchical reinforcement learning for self-driving decision-making without reliance on labelled driving data," *IET Intelligent Transport Systems*, 2020, 14(5): 297-305.
- [12] A. Vaswani *et al.*, "Attention Is All You Need," *arXiv:1706.03762*, 2017.
- [13] W. Xiao *et al.*, "Decision-Making for Autonomous Vehicles in Random Task Scenarios at Unsignalized Intersection Using Deep Reinforcement Learning," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 6, 2024, pp. 7812-7825.
- [14] D. Ha and J. Schmidhuber, "World Models," *arXiv:1803.10122*, 2018.
- [15] A. Hu *et al.*, "GAIA-1: A Generative World Model for Autonomous Driving." *arXiv:2309.17080*, 2023.
- [16] Z. Gao *et al.*, "Enhance Sample Efficiency and Robustness of End-to-End Urban Autonomous Driving via Semantic Masked World Model," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1-13, 2024.
- [17] L. Chen *et al.*, "Decision transformer: Reinforcement learning via sequence modeling," *Advances in neural information processing systems*, 2021, 34: 15084-15097.
- [18] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems." *arXiv:2005.01643*, 2020.
- [19] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
- [20] G. Konidaris and A. Barto, "Skill Discovery in Continuous Reinforcement Learning Domains using Skill Chaining," in *Advances in Neural Information Processing Systems*, 2009, pp. 1015-1023.
- [21] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Conference on robot learning*, PMLR, 2017, pp. 1-16.
- [22] Haoyang Wang, Chao Lu, Jianwei Gong, "Hierarchical reinforcement learning with successor representation for intelligent vehicle collision avoidance of dynamic pedestrian", *2024 9th International Conference on computer and communication Systems (ICCCS)*, IEEE, 2024: 575-581.